

6.4.2 基于案例的推理

基于案例的推理（CBR）分类器利用问题解决方案数据库来解决新问题。与将训练元组作为欧几里得空间中的点进行存储的k-最近邻分类器不同，CBR将用于解决问题的元组或“案例”存储为复杂的符号描述。CBR的商业应用包括客户服务支持中心的故障排除，其中案例描述了与产品相关的诊断问题。CBR还被应用于工程和法律等领域，其中案例分别指技术设计或普通法体系中的法律裁决。医学教育是CBR的另一个应用领域，其中利用患者的病史和治疗方案来帮助诊断和治疗新患者。

当需要对一个新案例进行分类时，基于案例的推理器会首先检查是否存在完全相同的训练案例。如果找到，则返回该案例对应的解决方案。如果未找到完全相同的案例，则基于案例的推理器会搜索那些与新案例的组成部分相似的训练案例。从概念上讲，这些训练案例可视为新案例的“邻居”。如果将案例表示为图，这相当于寻找与新案例内部子图相似的子图。基于案例的推理器会尝试将邻近训练案例的解决方案进行组合，从而为新案例提出解决方案。如果各解决方案之间出现冲突，则可能需要回溯以寻找其他解决方案。基于案例的推理器可能会运用背景知识和问题解决策略，以提出一个可行的综合解决方案。

基于案例的推理面临的主要挑战包括寻找合适的相似度度量（例如用于匹配子图）以及结合解决方案的合适方法。其他挑战还包括为训练案例选择突出的特征以进行索引，以及开发高效的索引技术。随着存储案例数量的激增，准确率与效率之间会产生权衡。随着案例数量的增加，基于案例的推理器会变得更加智能。然而，一旦超过某个临界点，由于搜索和处理相关案例所需的时间增加，系统的效率将受到影响。与最近邻分类器类似，一种解决方案是编辑训练数据库。为了提高性能，可以舍弃冗余的案例或那些已被证明无用的案例。然而，这些决策并非非黑即白，其自动化仍是一个活跃的研究领域。

6.5 线性分类器

到目前为止，我们已经学习了几个能够生成复杂决策边界的分类器。例如，决策树分类器可能会输出超矩形形状的决策边界（图 6.10(a)），而 k 最近邻分类器可能会输出超多边形形状的决策边界（图 6.10(b)）。然而，简单的线性决策边界又如何呢？以图 6.10 中的例子为例，直观来看，线性决策边界（图 6.10(c) 中的直线）在区分正样本和负样本方面，其效果（几乎）与决策树分类器和 k 最近邻分类器一样好。然而，这种线性决策边界可能还具有其他优势，例如分类器训练计算高效、泛化性能更优以及可解释性更强。

在本节中，我们将介绍学习此类线性分类器的基本技术。**我们将从线性回归开始**，它是线性分类器的基础。然后，我们将介绍两种线性分类器——

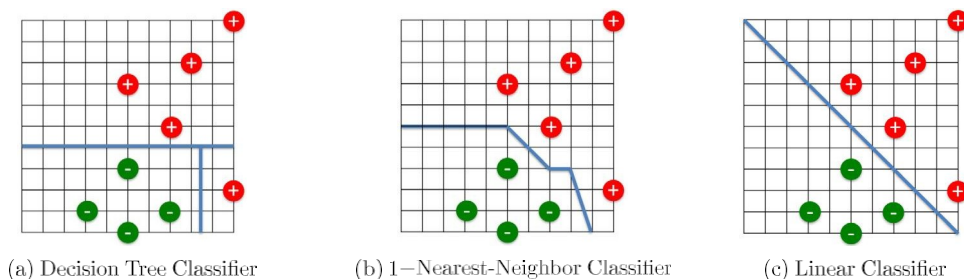


图 6.10

不同分类器绘制的决策边界。请注意，此示例为**线性可分**，这意味着线性分类器 (c) 能够将所有正训练元组与所有负训练元组完美分离。如果训练集为**线性不可分**，我们仍可使用线性分类器，但代价是部分训练元组会位于决策边界的“错误”一侧。在第7章中，我们将介绍处理线性不可分情况的技术（例如支持向量机）。

线性分类器，包括（1）**感知机（perception）**，这是最早的线性分类器之一，以及（2）**逻辑回归（logistic regression）**，这是应用最广泛的线性分类器之一。第7章还将介绍其他线性分类器，例如线性支持向量机。

6.5.1 线性回归

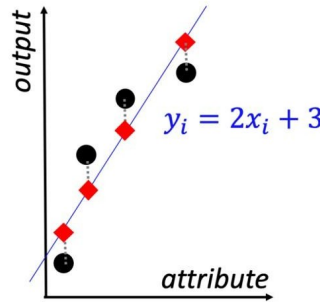
线性回归是一种基于一个或多个独立变量来预测连续值的统计技术。例如，我们可能希望根据居住面积预测房价，或者根据学生就读的大学、专业及总平均绩点等来预测其未来收入。由于线性回归旨在预测连续型值，因此无法直接应用于输出为分类变量的分类任务。尽管如此，线性回归的核心技术构成了线性分类器的基础。因此，让我们先简要介绍线性回归。

假设我们有 n 个元组，每个元组由 p 个属性 $x_i = (x_{i1}, \dots, x_{ip})^T$ 以及一个连续输出值 y_i ($i = 1, \dots, n$)。在线性回归中，我们希望学习一个线性函数，该函数将 p 个输入属性 x_i 映射到输出变量 y_i ，即 $\hat{y}_i = w^T x_i + b = \sum_{j=1}^p w_{(j)} \times x_{i,j} + b$ ，其中 y_i 是第 i 个元组的预测输出值， $w = (w_1, \dots, w_p)^T$ 是 p 维权重向量， b 是偏置标量。换句话说，线性回归假设输出值是 p 个输入属性值的线性加权和，并由偏置标量 b 进行偏移。权重向量 w_j ($j = 1, \dots, p$) 中的元素表明，在预测输出变量 $y_{(i)}$ 时，对应的属性 $x_{i,j}$ 具有多大的重要性。在前述示例中，线性回归模型会假设房价与居住面积呈线性相关；学生的未来收入可通过其就读的大学、专业以及总平均绩点（GPA）的线性加权组合（加上偏置标量 b ）来预测。如果我们知道权重向量 w 和偏置标量 b ，就可以根据 p 个输入属性值对输出值进行预测。

“那么，我们该如何确定权重向量 w 和偏置标量 b 呢？”直观地讲，我们希望从训练数据中学习到“最佳”的权重向量 w 和“最佳”的偏置标量 b ，从而使线性——

Index (i)	1	2	3	4
Attribute (x_i)	1	3	5	7
Output (y_i)	4	10	14	16

(a) Training tuples



(b) Least square regression

图 6.11

最小二乘回归的一个示例。(a) 四个训练元组。(b) 训练元组（黑色圆点）与最小二乘回归模型（蓝色直线）的散点图。红色菱形表示预测输出 $\hat{y}_i$ ($i = 1, 2, 3, 4$)，虚线表示相应训练元组的预测误差 ($|y_i - \hat{y}_i|$)。

线性回归模型能够做出“最佳”预测。也就是说，预测值 $y_i = w^T x_i + b$ 应尽可能接近实际观测值 y_i ($i = 1, \dots, n$)。最常见的线性回归方法之一称为**最小二乘回归**，其目标是将以下损失函数最小化——函数 $L(w, b) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (w^T x_i + b))^2$ 。因此，最优权重向量 w 和偏置标量 b 是使损失函数 $L(w, b)$ ，该函数衡量预测输出值 $\hat{y}_i$ 与实际观测值 y_i 之间的平方差。例如——

例如，若仅有一个输入属性（即 $p = 1$ ），则最优权重 $w =$

$$\text{最优偏置标量 } b = \frac{1}{n} \sum_{i=1}^n (y_i - w x_i), \text{ 其中 } \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i \text{ 是观测到的平均输出值}$$

在所有 n 个训练元组中。

例 6.7. 让我们通过图 6.11 来看一个最小二乘回归的例子。这里有四个训练元组，每个元组由一个一维属性 x_i 和一个输出变量 y_i 表示 ($i \in 1, 2, 3, 4$)。我们希望找到一个最小二乘回归模型 $y = wx + b$ ，该模型能够根据输入属性 x 预测输出 y 。我们使用上述两个方程来求解最优权重 w 和最优偏置标量 b 。首

先，我们按如下方式求解最优权重 w 。平均输出——

四个训练元组的输出为 $\bar{y} = (y_1 + y_2 + y_3 + y_4)/4 = (4 + 10 + 14 + 16)/4 = 11$ 。因此我们得到 $\sum_{i=1}^4 x_i (y_i - \bar{y}) = 1(4 - 11) + 3(10 - 11) + 5(14 - 11) + 7(16 - 11) = 40$ 。在平均同时，我们有 $\sum_{i=1}^4 x_i^2 = 1^2 + 3^2 + 5^2 + 7^2 = 84$ ，且 $1/4 \sum_{i=1}^4 x_i^2 = (1 + 3 + 5 + 7)^2 / 4 = 64$ 。

$$\text{因此，最优权重 } w = \frac{\sum_{i=1}^n x_i (y_i - \bar{y})}{\sum_{i=1}^n x_i^2 - \frac{1}{n} (\sum_{i=1}^n x_i)^2} = \frac{40}{84 - \frac{1}{4}(1+3+5+7)^2} = \frac{40}{84 - 64} = \frac{40}{20} = 2.$$

基于最优权重 w ，该最优偏置标量 $b = \frac{1}{n} \sum_{i=1}^n (y_i - w x_i) = \frac{1}{4} (4 - 2 \times 1) + \frac{1}{4} (10 - 2 \times 3) + \frac{1}{4} (14 - 2 \times 5) + \frac{1}{4} (16 - 2 \times 7) = \frac{1}{4} (2 + 4 + 4 + 4) = 3.$

□

“但是，如果存在多个 p ($p > 1$) 个属性呢？”在这种情况下（称为**多重线性回归**），让我们先稍微调整一下记号。我们假设存在一个额外的“虚拟”属性，该属性对于任何元组都始终取值为 1。设该虚拟属性的权重

的权重为 w_0 。此时，总权重向量 $w = (w_0, w_1, \dots, w_p)$ 以及新的输入属性向量 $x_i = (1, x_{i,1}, \dots, x_{i,p})$ 均为 $(p+1)$ 维向量。多线性回归模型可表示为

重写为 $y_i = w^T x_i = w_0 + w_1 x_{i,1} + \dots + w_p x_{i,p}$ 。我们使用与之前相同的损失函数，即

$$L(w) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (w^T x_i))^2$$

事实证明，最优权重向量 w 可以由

$$w = (X^T X)^{-1} X^T y$$

被视为 $w = (X^T X)^{-1} X^T y$ ，其中 $X = [x_1, x_2, \dots, x_n]$ 是一个 $(p+1) \times n$ 矩阵，且 $y = [y_1, \dots, y_n]^T$ 是一个 $n \times 1$ 的向量。（如何推导单变量线性回归以及多重线性回归的闭式解推导留作习题。）

在最小二乘回归中，我们通过预测值与实际输出值之间差值的平方和来衡量所学习回归模型的“拟合优度”。平方误差可能对训练集中的异常值较为敏感。在**鲁棒回归**中，则采用对这类异常值不那么敏感的替代损失函数。例如，鲁棒回归中的 Huber 方法

使用以下损失函数： $L(w) = \sum_{i=1}^n l_H(y_i - \hat{y}_i)$ ，其中 $l_H(y_i - \hat{y}_i) = (y_i - \hat{y}_i)^2$ 若 $|y_i - \hat{y}_i| < \theta$ ， $l_H(y_i - \hat{y}_i) = 2\theta |y_i - \hat{y}_i| - \theta^2$ 否则，以及 $\theta > 0$ 是用户指定的参数。请注意，opti-

多线性回归的优化权重向量 w 涉及矩阵逆运算（即 $(X^T X)^{-1}$ ）。当 $p > n$ （即属性数多于训练元组数）时，该矩阵逆运算

不存在。解决这一问题的有效方法是引入一个与权重向量 w 的范数相关的**正则化项**。例如，如果我们使用 l_2 作为权重向量 w 的范数，相应的回归模型称为 Ridge 回归；如果我们改用 l_1 作为权重向量 w 的范数，相应的回归模型则称为 Lasso 回归，该模型通常会学习到一个**稀疏**的权重向量。这意味着所学习到的权重向量 w 的某些元素为零，这表明这些特征在回归模型中未被使用。在第 7.1 节中，我们将使用 Lasso 回归进行特征选择。

6.5.2 感知器：将线性回归转化为分类

“如何修改线性回归模型以执行分类任务？”假设我们有一个二分类任务。¹⁰ 对于给定的元组，其输出值 y_i 是一个二元变量： $y_i = 1$ 表示第 i 个元组是正类元组（例如“购买电脑”），而 $y_i = 0$ 表示第 i 个元组是负类元组（例如“不购买电脑”）。针对此类二元分类任务，修改线性回归模型的一种方法是将线性回归模型输出的符号作为预测的类别标签，即 $y_i = \text{sign}(w^T x_i)$ ，其中 y_i 是第 i 个元组的预测类别标签， $\text{sign}(z)$ 取值为 1 若 $z > 0$ ，否则取值为 0。请注意，我们采用了与多重线性回归相同的记法，其中引入了一个“虚拟”属性，该属性对任何元组始终取值为 1。因此，如果已知权重向量 w ，我们可以按如下方式利用它来预测给定元组的类别标签。我们计算给定元组各属性值的线性组合，权重由权重向量 w 的对应分量决定。若该线性组合的计算结果为正值，则预测该元组为正元组；否则，预测其为负元组。

“如何从一组训练元组中找到最优权重向量 w ？”训练感知器的经典学习算法如下。我们首先对权重向量 w 进行初始猜测（例如，

¹⁰ 对于接下来将介绍的感知机和逻辑回归分类器，我们将重点关注二分类任务。不过，我们介绍的技术可以推广到处理这两种分类器的多分类任务。

我们可以简单地将 w 设为0。然后，学习算法将迭代直至收敛，或者达到最大迭代次数或其他预设的终止条件。在每次迭代中，对于每个训练元组 x_i ，我们执行以下操作：我们尝试使用当前权重向量 w 来预测 x_i 的类标签，即 $y_i = \text{sign}(w^T x_i)$ 。如果预测正确（即 $y_i = y_i$ ），则不对权重向量进行任何调整。然而，如果预测错误（即 $y_i \neq y_i$ ），我们将通过以下两种方式之一更新当前权重向量。如果 $y_i = 1$ （即第 i 个元组是正元组，但当前分类器错误地预测其为负元组），则将权重向量更新为 $w \leftarrow w + \eta x_i$ 。若 $y_i \neq 0$ （即第 i 个元组为负元组，但当前分类器错误地将其预测为正元组），则将权重向量更新为 $w \leftarrow w - \eta x_i$ ，其中 $\eta > 0$ 是用户指定的学习率。因此，其核心思想在于：在训练过程的每次迭代中，算法将重点关注当前权重向量 w 预测错误的训练元组。若预测错误的训练元组 x_i 属于正元组，则通过将其向量向该训练元组的属性向量 x_i 方向移动来更新权重向量 w （即 $w \leftarrow w + \eta x_i$ ）。另一方面，如果预测错误的训练元组 x_i 是一个负元组，则我们通过将权重向量 w 远离该训练元组的属性向量 x_i 来更新 w （即 $w \leftarrow w - \eta x_i$ ）。

例 6.8. 让我们来看图 6.12 中一个关于训练感知机分类器的例子。在图 6.12 中，为了说明的清晰起见，我们假设偏置项 $w_0 = 0$ 。图 6.12(a)（左）显示了当前的决策边界和权重向量 w ，其中有两个训练元组被错误分类，包括一个正元组 x_1 和一个负元组 x_8 。因此，在当前迭代中仅使用这两个元组来更新权重向量，即 $w \leftarrow w + \eta x_1 - \eta x_8$ 。图 6.12(b)（右）显示了更新后的权重向量 w 及相应的决策边界，此时所有训练元组均被正确分类。

“感知机学习算法的有效性如何？”如果训练元组是线性可分的（例如图 6.12 中的示例），则感知机算法必定能找到一个权重向量（即超平面决策边界），该向量能将所有正训练元组与所有负训练元组完美地分离。然而，如果训练元组不可线性分离，该算法将无法收敛。

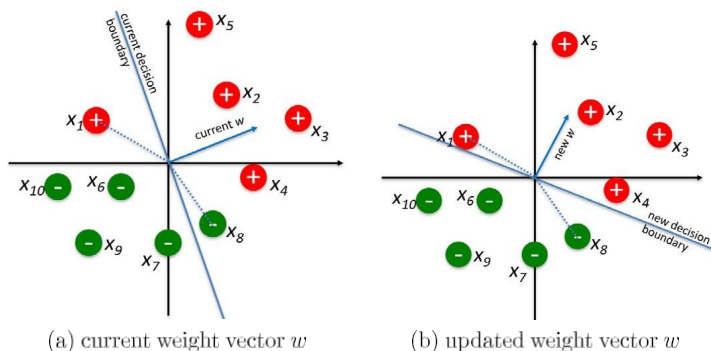


图 6.12

训练感知机分类器。

感知器是最早的线性分类器之一，最早发明于 1958 年。它也可以作为第 10 章将介绍的神经网络中的一个构建模块（称为“神经元”）。

6.5.3 逻辑回归

上一节中刚介绍的感知机能够预测给定元组的二元类标签。然而，我们能否同时判断该预测的置信度呢？让我们再次考虑一个二元分类任务，并假设存在两种可能的类标签，即正元组对应 $y = 1$ ，负元组对应 $y = 0$ 。回顾（朴素）贝叶斯分类器，我们可以估计后验概率 $P(y_i | x_i)$ ，该概率可直接用于表明预测分类结果的置信度。例如，若 $P(y_i | x_i)$ 接近 1，则分类器对元组 x_i 属于正样本具有高度信心。

如何让线性分类器不仅能预测元组的类别标签，还能说明其对该预测的置信度？实现这一目标的有效方法是使用逻辑回归分类器。首先，我们介绍一个名为 sigmoid 函数的重要函数，其定义为 $\sigma(z) = \frac{1}{1+e^{-z}}$ 。从图 6.13 中可以看出，sigmoid 函数将一个实数映射到 $(-\infty, +\infty)$ （即图 6.13 的 x 轴）映射到 $(0, 1)$ 范围内的输出值（即图 6.13 的 y 轴）。因此，如果我们利用 sigmoid 函数将线性回归模型的输出映射到 0 到 1 之间的数值，就可以将映射结果解释为观察到正类标签的后验概率。这正是逻辑回归分类器试图做的事情！

从形式上讲，我们有 $P(y_i = 1 | x_i, w) = \sigma(w^{(T)} \times x_i) = \frac{1}{1+e^{-w^{(T)} \times x_i}}$ ，其中 y_i 是该元组具有属性 x_i ， w 是权重向量。请注意，为了简化记法，我们引入了一个虚拟属性，将偏置项 b 合并到了权重向量 w 中，这与我们在多线性回归模型和感知器中的做法一致。显然，若 $P(y_i = 1 | x_i, w) > 0.5$ ，则分类器预测元组 x_i 为正元组（即 $y_i = 1$ ）；否则，预测为负元组（即 $y_i = 0$ ）。这（细节留作习题 2）等同于以下线性分类器：若 $w^{(T)} \times x_i > 0$ ，则预测 $y_i = 1$ （即正元组）；若 $w^{(T)} \times x_i < 0$ ，则预测 $y_i = 0$ （即负元组）。因此，如果已知权重向量 w ，对给定元组的分类任务就非常简单。也就是说，我们

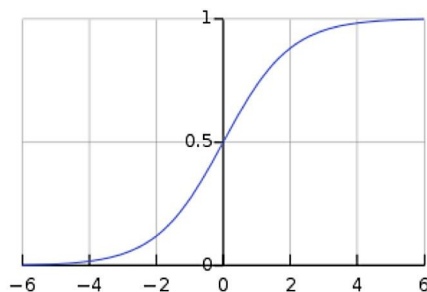


图 6.13

正弦函数的示意图。正弦函数将输入从较大的范围 $(-\infty, +\infty)$ “压缩”到较小的范围 $(0, 1)$ 。因此，正弦函数也被称为压缩函数。在第 10 章中，我们将看到

其他类型的压缩函数，在深度学习术语中，这些被称为激活函数。

只需将给定元组的属性向量 x_i 与权重向量 w 相乘，然后根据 $w^T \times x_i$ 的符号进行预测。如果 $w^T \times x_i$ 为正数，则预测该元组为正元组；否则，预测其为负元组。

“如何从一组训练元组中确定最优权重向量 w ？”训练逻辑回归分类器（即从训练集确定最佳权重向量 w ）的经典方法是通过最大似然估计（MLE）。再次假设存在 n 个训练元组 (x_i, y_i) ($i = 1, \dots, n$)。由于这是一项二分类任务，我们可以将预测的类标签 $\hat{y}_i$ 视为伯努利随机变量，其仅可能取两个值，即 $P(\hat{y}_i = 1 | x_i, w) = p_i$ 以及 $P(\hat{y}_i = 0 | x_i, w) = 1 - p_i$ ，其中 $p_i = \sigma(w^T \times x_i) = \frac{1}{1 + e^{-w^T \times x_i}}$ 由

S型函数，它描述了预测类标签（即 $y_i \in \{0, 1\}$ ）出现正结果的概率。请注意，第 i 个元组的真实类标签 y_i 是一个二元变量。因此

我们有 $P(y_i = y_i) = p_i^{y_i} (1 - p_i)^{1-y_i}$ 。最大似然估计法旨在求解以下优化问题，该问题要求我们选择最佳权重向量 w ，使得

最大化训练集的似然函数。其直观含义是，我们希望找到最优的模型参数（即权重向量 w ），使得观测到整个训练集的“可能性”（即似然或概率）达到最高。

$$w^* = \underset{w}{\operatorname{argmax}} L(w) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i} = \prod_{i=1}^n \left(\frac{e^{w^T x_i}}{1 + e^{w^T x_i}} \right)^{y_i} \left(\frac{1}{1 + e^{w^T x_i}} \right)^{1-y_i} \quad (6.19)$$

“但是，我们该如何设计一个算法来解决这个优化问题，从而求得最优权重向量 w 呢？”首先，我们注意到似然函数 $L(w)$ 包含许多相互相乘的非负项。在实际应用中，处理此类复杂函数的对数通常更为方便。因此，我们得到以下等价的优化问题，其中 $l(w)$ 被称为对数似然

$$w^* = \underset{w}{\operatorname{argmax}} l(w) = \sum_{i=1}^n y_i x_i^T w - \log(1 + e^{w^T x_i}) \quad (6.20)$$

从优化角度来看，好消息是式(6.20)中的对数似然函数是一个严格凹函数，因此其极值（最优解）唯一存在。然而，坏消息是上述优化问题不存在闭式解。在

在这种情况下，一种常见的策略是按照以下方式迭代求解最优解 w^* 。在每次迭代中，我们试图改进当前的权重向量 w ，以便最大化我们希望优化的目标函数

（对数似然函数 $l(w)$ ）得到最大程度的提升。为了使当前目标函数 $l(w)$ 增加最多，事实证明，更新权重向量 w 当前估计值的最佳方向是沿着其梯度方向进行。这导致了以下算法，用于从训练集学习最优权重向量 w^*

。我们从权重向量 w 的初始猜测开始（例如，我们可以简单地设置 $w \equiv 0$ ）。随后，学习算法将进行迭代，直至收敛，或达到最大迭代次数，或满足其他预设的终止条件。在每次迭代中，我们将权重向量 w 更新为如下： $w \leftarrow w + \eta \sum_{i=1}^n (y_i - P(\hat{y}_i = 1 | x_i, w)) x_i$ ，其中 $\eta > 0$ 是用户指定的学习率。

“那么，上述算法的直观理解是什么？”让我们分析每次训练的影响元组 (x_i, y_i) 在更新权重向量 w 的估计值时。我们根据该元组是正元组（即 $y_i = 1$ ）还是负元组（即 $y_i = 0$ ）来考虑两种情况。对于前者，

给定元组对权重向量 w 的更新影响可计算为 $w \leftarrow w + \eta(1 - P(\hat{y}_i = 1|x_i, w))x_i$ 。其直观含义是，我们希望将当前权重向量 w 向该正元组的属性向量 x_i 的方向进行更新。对于后一种情况（即 $y_i = 0$ ），给定元组对权重向量 w 的更新影响可计算为 $w \leftarrow w - \eta P(\hat{y}_i = 1|x_i, w)x_i$ 。其直观含义是

我们希望将当前权重向量 w 向远离该负样本的属性向量 x_i 的方向进行更新。从这一角度来看，用于训练逻辑回归分类器的学习算法与感知机算法存在某些相似之处。也就是说，这两种算法都试图更新当前权重向量 w ，使其 (1) 与正样本元组的属性向量更加一致，以及 (2) 与负样本元组的属性向量更加不一致（即朝向相反方向）。

然而，这两种算法（感知机与逻辑回归）在权重向量 w 的更新幅度上存在差异。在感知机中，对于所有错误的

由当前权重向量 w 预测的元组。另一方面，在逻辑回归中，它不仅取决于学习率 η ，还取决于 $P(\hat{y}_i = 1|x_i, w)$ （即基于当前权重向量 w ，给定元组属于正类别的概率）。这使得逻辑回归算法

在以下意义上是自适应的。例如，如果对于某个正样本， $P(\hat{y}_i = 1|x_i, w)$ 的值很高，这意味着当前权重向量 w 对该正样本的预测不仅正确（即 $P(\hat{y}_i = 1|x_i, w) > 0.5$ ），而且信心程度很高（即 $P(\hat{y}_i = 1|x_i, w)$ 接近 1）。因此，该正元组对权重向量更新的影响（即 $\eta(1 - P(\hat{y}_i = 1|x_i, w))$ ）相对较小。另一方面，如果对于一个负元组， $P(\hat{y}_i = 1|x_i, w)$ 很高，这意味着当前权重向量 w 对该负元组的预测要么是错误的（即 $P(\hat{y}_i = 1|x_i, w) > 0.5$ ），或者虽然正确但置信度较低（即 $P(\hat{y}_i = 1|x_i, w)$ 仅略低于 0.5）。此时，该负元组（即 $\eta P(\hat{y}_i = 1|x_i, w)$ ）对权重向量更新的影响将相对较大。换言之，逻辑回归

回归学习算法更关注那些“困难”的训练元组，即当前权重向量 w 预测错误或虽预测正确但置信度较低的元组。回顾图 6.12(a) 中的示例，由于当前的 w 将这两个元组分类错误，因此感知机仅使用 x_1 和 x_8 来更新当前权重向量 w 。相比之下，逻辑回归会利用所有训练元组来更新权重向量 w 。其中， x_1 和 x_8 对更新 w 的影响最大，因为它们都被当前分类器错误分类； x_2 、 x_3 、 x_5 、 x_9 和 x_{10} 的影响最小，因为它们均被当前权重向量 w 以高置信度正确分类； x_4 、 x_6 和 x_7 的影响适中，因为它们虽被正确分类，但置信度相对较低。

“逻辑回归算法的性能如何？它有哪些潜在局限性，又该如何缓解？”由于对数似然函数 $l(w)$ 是一个凹函数，因此上述用于训练逻辑回归分类器的算法必然收敛于其最优解。然而，如果训练集是线性可分集，该算法可能会收敛到一个范数无限大的权重向量 w 。（参见图 6.14 中的示例。）“庞大”的权重向量 w 可能导致训练后的分类器容易受到给定元组中某些属性的噪声影响。这进而会导致所学逻辑回归分类器的泛化性能不佳。换言之，所学逻辑回归分类器对训练集产生了过拟合。缓解过拟合的一种有效方法是在目标函数 $l(w)$ 中引入正则化项 $\|w\|^2$ ，以防止所学到的权重向量

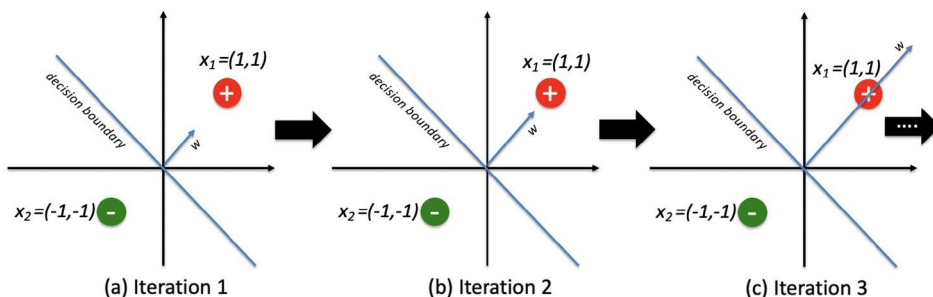


图 6.14

线性可分情况下逻辑回归权重向量无限大的示意图。二维空间中有两个训练元组，一个正训练元组 $x_1 = (1, 1)$ ，一个负训练元组

$x_2 = (-1, -1)$ 。为简化起见，设偏置标量 $b = 0$ ，学习率 $\eta = 1$ 。假设在第

1，权重向量 $w = (1, 1)$ 。那么，上述介绍的逻辑回归算法将更新权重向量为 $w_{\text{new}} = w_{\text{old}} + (1 - P(\hat{y}_1 = 1|x_1, w_{\text{old}}))x_1 - P(\hat{y}_2 = 1|x_2, w_{\text{old}})x_2 = (0.5, 0.5) + a(1, 1) = (1 + 2a)w_{\text{old}}$ ，其中 $a = 1 - P(\hat{y}_1 = 1|x_1, w_{\text{old}}) + P(\hat{y}_2 = 1|x_2, w_{\text{old}}) > 0$ 。因此，新权重向量 w_{new} 与旧权重 w_{old} 方向一致。因此，决策边界保持不变，但新权重向量 w_{new} 的模值会增加 $(1 + 2a)$ 倍。随着逻辑回归算法的进行，这种趋势将持续下去，最终导致权重向量的模值无限增大。

变得“过大”。¹¹ 第二个潜在局限性在于逻辑回归背后的独立性假设。回顾一下，当我们计算训练集的似然函数 $L(w)$ 时，我们只是将每个训练元组的似然值相乘（式(6.19)）。这意味着我们隐含地假设了不同的训练元组彼此独立。然而，在某些应用中，这一假设可能会被违反（例如，社交网络上的用户彼此相互关联）。基于图的

分类可能为这一问题提供了一种自然的解决方案。第三个潜在局限性在于

计算挑战。请注意，在更新规则 $w \leftarrow w + \eta \sum_{i=1}^n (y_i - P(\hat{y}_i = 1|x_i, w))x_i$

中，我们需要计算梯度 $(y_i - P(\hat{y}_i = 1|x_i, w))$ 所有训练元组，并

然后将它们求和以更新权重向量 w 。如果训练元组有数百万个，进行此类计算的计算成本将非常高。解决此问题的一种有效方法是使用随机梯度下降法来训练逻辑回归分类器。也就是说，在每次迭代中，我们将从训练元组中随机抽取一个小子集（这通常被称为“小批量”），并仅使用这些抽取的元组（而非所有训练元组）来更新权重向量。值得指出的是，随机梯度下降法在许多其他数据挖掘算法中被广泛应用，例如深度学习方法，相关内容将在第10章中介绍。

¹¹ 从统计参数估计的角度来看，我们正从最大似然估计（MLE）转向最大后验估计（MAP）。在 $L(w)$ 中加入正则化项 $\|w\|^2$ ，相当于为权重向量 w 施加一个高斯先验分布

，其均值向量位于原点，用于权重向量 w_0 。